

ADE-Bot

Documentacion del Proyecto

Chatbot institucional para el Area de Ingenieria en Diseno

Universidad | Ingeniería en Diseño | 2026

1. Descripcion del Proyecto

ADE-Bot es un chatbot institucional desarrollado para el area de Ingenieria en Diseno. Responde consultas sobre el pensum, tramites administrativos, licitaciones, normativas de diseno, presupuesto y seguridad en obras, utilizando unicamente informacion de los documentos oficiales del area.

El bot esta disponible a traves de una interfaz web moderna con visor de materiales 3D integrado.

Objetivos

- Centralizar el acceso a informacion institucional del area ADE en un solo canal conversacional.
- Reducir la carga de consultas repetitivas al personal administrativo.
- Garantizar precision mayor o igual al 90% en las respuestas mediante un banco de pruebas controlado.
- Facilitar la consulta de materiales y normativas con modelos 3D interactivos.

Tecnologias Utilizadas

Python 3.11+	FastAPI	LangChain	ChromaDB	Google Gemini	Groq / Llama	SQLite
Tailwind CSS	Sketchfab					

2. Arquitectura del Sistema

El proyecto combina dos patrones complementarios:

RAG

Arquitectura en Capas

RAG (Retrieval-Augmented Generation)

Es el patron principal de inteligencia artificial. Cada consulta pasa por tres etapas antes de llegar al modelo de lenguaje:

- 1. Recuperacion: la consulta se convierte en un vector de embeddings y se buscan los fragmentos de documentos con mayor similitud semantica en ChromaDB.
- 2. Aumentacion: los fragmentos recuperados se concatenan como contexto al prompt del sistema.
- 3. Generacion: el modelo de lenguaje (Groq o Gemini) genera una respuesta fundamentada exclusivamente en ese contexto.

Esto evita que el bot alucine informacion: si no hay fragmentos relevantes, lo indica y responde desde conocimiento general.

Arquitectura en Capas

El codigo se organiza en tres capas con responsabilidades claras:

Capa	Carpeta	Responsabilidad
Presentacion	main.py, static/	Endpoints REST e interfaz web. Sin logica de negocio.
Servicios	services/	Orquesta la busqueda, construye el prompt y coordina el log.
Infraestructura	infrastructure/	Implementaciones concretas: ChromaDB, LLM y SQLite.

Cada capa depende de interfaces abstractas, no de implementaciones concretas. Esto permite sustituir cualquier componente sin modificar la logica de negocio.

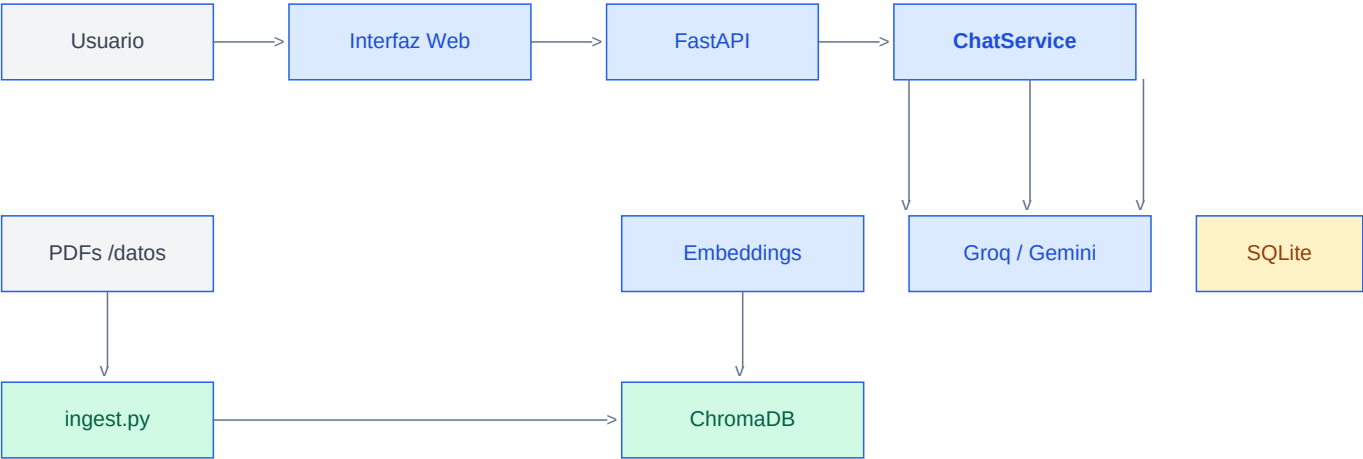
Pipeline de Ingesta (Offline)

Los documentos PDF del area se procesan una sola vez para indexarlos en ChromaDB. No forma parte del servidor en ejecucion:

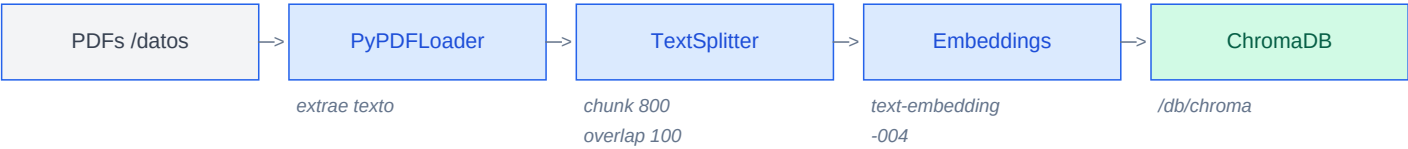
```
PDF en /datos
-> PyPDFLoader (extrae texto por pagina)
-> RecursiveCharacterTextSplitter (chunk=800, overlap=100)
-> Asignacion de metadata (tema y fuente por nombre de archivo)
-> GoogleGenerativeAIEmbeddings (text-embedding-004)
-> ChromaDB en /db/chroma
```

3. Diagramas

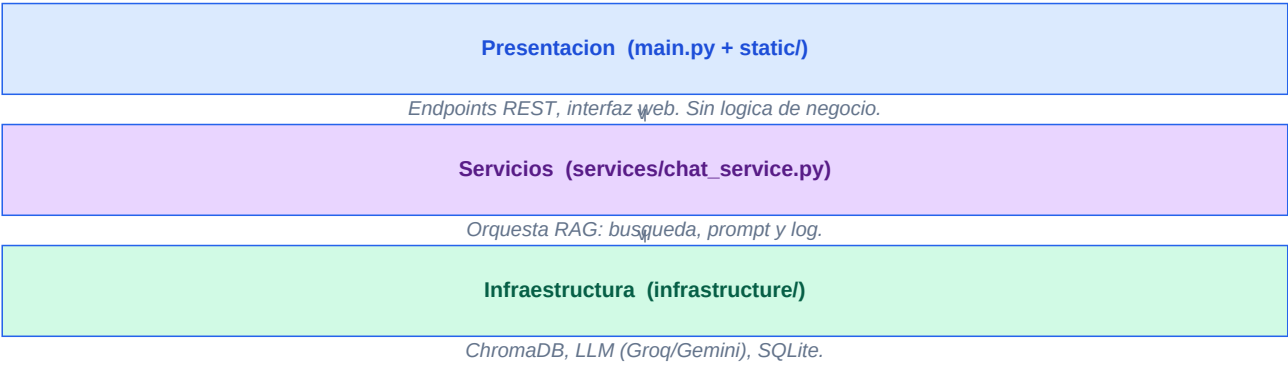
Diagrama general del sistema



Pipeline de ingesta (offline)



Arquitectura en capas



4. Equipo y Modulos

Modulo	Nombre	Area de Responsabilidad
1	Luis Millan	Base de Conocimientos: inventario y segmentacion de 6 documentos
2	Arnaldo Perdomo	IA y PLN: configuracion del LLM y glosario tecnico
3	Yvanna Bravo	Interfaz Web: diseno visual de la aplicacion web
4	Keibel Guilarte	Pruebas y Control de Calidad: banco de preguntas y criterio 90%
5	Alejandro Duarte	Infraestructura y Soporte: hosting, logs y despliegue

Cronograma

Modulo	Actividad	Entregable	Semana
1	Inventario y segmentacion documental	6 documentos limpios	1
2	Configuracion IA y flujos de intencion	Glosario y logica optimizada	3
3	Diseno e implementacion de la interfaz web	Interfaz web funcional	4
4	Pruebas de estres y control de errores	Reporte de depuracion >= 90%	5
5	Hosting definitivo, difusion y logs	Despliegue en produccion	7
-	Entrega formal al profesor	Documento definitivo (PDF)	20/05/2026

5. Estructura del Proyecto

```
ADE-Bot Project/
+-- main.py          Servidor FastAPI y endpoints REST
+-- services/
|   |-- chat_service.py    Logica RAG: busqueda, prompt, respuesta y log
+-- infrastructure/
|   +-- llm_client.py      Clientes LLM: GeminiLLMClient y GroqLLMClient
|   +-- vector_store.py    ChromaVectorStore con embeddings de Google
|   |-- log_repository.py  Registro de interacciones en SQLite
+-- ingesta/
|   |-- ingest.py          Pipeline offline: PDF a ChromaDB
+-- tests/
|   |-- banco_preguntas.py Banco de pruebas del Modulo 4
+-- static/
|   |-- index.html         Interfaz web (chat y visor 3D)
+-- datos/               PDFs fuente (no versionado)
+-- db/                  ChromaDB y SQLite (generados en ejecucion)
+-- docs/                Documentacion del proyecto
+-- .env                 Variables de entorno (no versionado)
+-- .env.example          Plantilla de configuracion
+-- requirements.txt      Dependencias Python
```

6. Guía de Instalacion

Requisitos previos: Python 3.11+, Git, clave GEMINI_API_KEY (obligatoria) y opcionalmente GROQ_API_KEY.

Paso 1: Clonar o descargar el repositorio

```
git clone <url-del-repositorio>
cd "ADE-Bot Project"
```

Paso 2: Crear el entorno virtual

```
python -m venv .venv
```

Paso 3: Activar el entorno virtual

```
.venv\Scripts\activate          # Windows
# source .venv/bin/activate      # macOS/Linux
```

Paso 4: Instalar dependencias

```
pip install -r requirements.txt
```

Paso 5: Configurar variables de entorno

```
copy .env.example .env
# Abrir .env y pegar las claves API
```

Paso 6: Indexar los documentos PDF (primera vez)

```
# Colocar los PDFs del area en la carpeta datos/
python -m ingesta.ingest
```

Paso 7: Levantar el servidor

```
python main.py
# La interfaz estara disponible en http://localhost:8000
```

7. API REST

Metodo	Endpoint	Descripcion
GET	/	Sirve la interfaz web (index.html)
POST	/api/chat	Envia consulta al bot, recibe respuesta JSON
GET	/api/fuentes	Lista PDFs adjuntos a una sesion
POST	/api/upload	Adjunta un PDF a la sesion activa
DELETE	/api/upload	Elimina un PDF adjunto de la sesion

Ejemplo: POST /api/chat

```
curl -X POST http://localhost:8000/api/chat \  
  -H "Content-Type: application/json" \  
  -d '{"mensaje": "cuantos creditos tiene estructuras", "session_id": "abc123"}'  
  
# Respuesta:  
{"respuesta": "La materia Estructuras I tiene 4 creditos segun el pensum vigente."}
```

Los documentos de sesion se almacenan en memoria y se pierden al reiniciar el servidor.

La API no requiere autenticacion en entorno de desarrollo.

8. Interfaz Web

La interfaz (static/index.html) es una aplicacion web de pagina unica construida con Tailwind CSS y JavaScript vanilla. No requiere framework adicional.

Componentes principales

Componente	Descripcion
Panel de chat	Conversaciones multiples con historial en la barra lateral izquierda
Visor 3D	Modelos Sketchfab de 14 materiales de ingenieria, activados automaticamente
Subida de PDF	Adjunta un documento a la sesion activa para ampliar el contexto del bot
Historial	Las ultimas sesiones se guardan en localStorage del navegador
Materiales activos	Al mencionar un material en la consulta, el visor lo selecciona en la barra

Materiales disponibles en el visor 3D

Material	Material	Material	Material
Acero Inoxidable	Aluminio	Titanio	Cobre
Fibra de Carbono	Polimero	Ceramica	Concreto
Madera	Vidrio	Hormigon	Bronce
Plastico ABS	Acero al Carbon		

9. Pruebas y Control de Calidad

Responsable: Keibel Guilarte | Criterio de aprobacion: >= 90% de precision

Banco de preguntas

Categoria	Cantidad	Descripcion
Consultas institucionales	8	El bot debe encontrar contexto y responder
Fuera de alcance	4	El bot debe reconocer que no corresponde al area

Casos de prueba

Consulta	Resultado esperado
oye cuantos creditos tiene estructuras	Resolver
pa pedir un permiso que hago	Resolver
que necesito pa inscribir la materia	Resolver
que vaina necesito pa la licitacion	Resolver
el blueprint ese como se llama	Resolver
reglamento de las obras	Resolver
informacion del area	Resolver
cuanto cuesta el tramite	Resolver
ayudame con mi tesis	Rechazar
cual es la capital de venezuela	Rechazar
como programo en python	Rechazar
dimelo en ingles	Rechazar

Ejecutar el banco

```
python -m tests.banco_preguntas
```

Como funcionan los dobles de prueba

El banco usa fakes para los tres componentes del ChatService: `_FakeVectorStore`, `_FakeLLMClient` y `_FakeLogRepository`. Esto permite correr las pruebas sin conexion a Gemini ni a ChromaDB, evitando consumo de cuota.

10. Seguridad

Aspecto	Medida implementada
Claves API	Almacenadas solo en .env, excluido de git via .gitignore
Archivos sensibles	datos/ y db/ ignorados por git
Entrada del usuario	Texto enviado como JSON, sin ejecucion de comandos
Carga de archivos	Validacion de extension .pdf antes de procesar
Sesiones	session_id generado aleatoriamente por el frontend

11. Dependencias

Paquete	Version minima	Uso
fastapi	0.115.0	Framework web y definicion de endpoints REST
uvicorn	0.30.0	Servidor ASGI para FastAPI
langchain	0.3.0	Orquestacion del pipeline RAG
langchain-google-genai	2.0.0	Cliente Gemini (embeddings y LLM)
langchain-groq	-	Cliente Groq (LLM alternativo)
langchain-chroma	0.1.4	Integracion ChromaDB con LangChain
chromadb	0.5.23	Base de datos vectorial local
pypdf	4.2.0	Extraccion de texto de archivos PDF
python-dotenv	1.0.1	Carga de variables de entorno desde .env
python-multipart	0.0.9	Soporte para carga de archivos (multipart)
langchain-text-splitters	0.3.0	Division de documentos en fragmentos